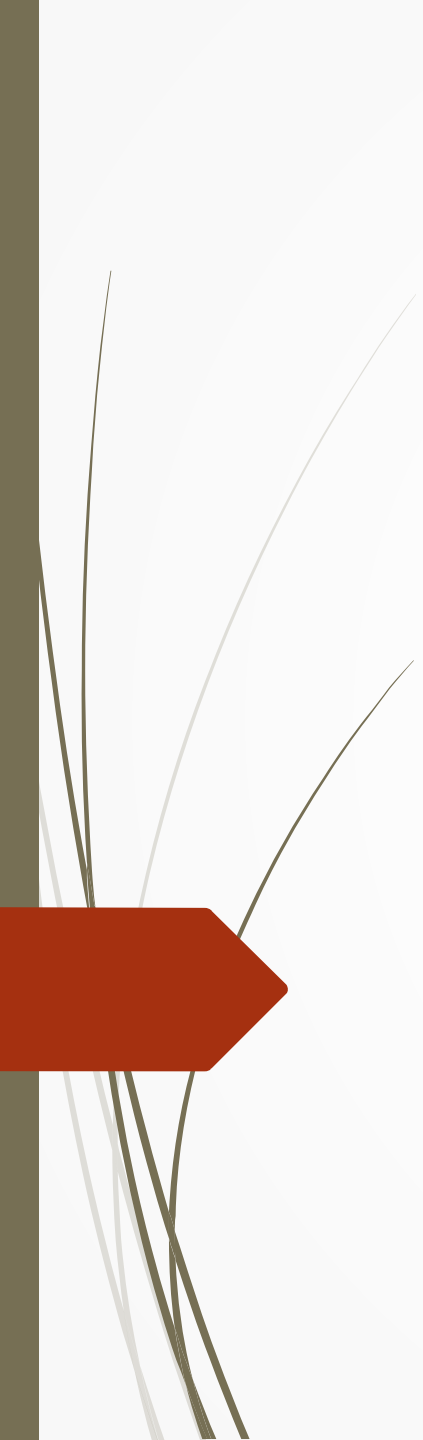




CS1004 – Object Oriented Programming

Lecture # 07

Monday, February 28, 2022

SPRING 2022

FAST – NUCES, Faisalabad Campus

Muhammad Yousaf

Structures

2

- A structure is user defined data type
- It provides data abstraction
- It is a collection of different type of data types

Student:

Roll_No	Name	CNIC	Date_of_Birth
---------	------	------	---------------

Example:

21F-1234	Ali Raza	12345-1234567-1	1/1/2000
----------	----------	-----------------	----------

Declaring struct in C++

3

➤ Syntax

```
struct struct_name
{
    type member1;
    type member2;
    type member3;
    . . . . .
};
```

Example: struct in C++

4

```
struct Time
{
    int hours;
    int minutes;
    int seconds;
};
```



Members/Attributes/Fields

- This declares a new datatype '**Time**'
- We can use this to create variables of type '**Time**'
- **Note:** '**Time**' is not a variable. There is no memory allocated for it. It is just a user defined datatype just like primitive datatypes i.e. **int**, **char** etc.

Defining structure variable

5

➤ Syntax

```
struct_name variable_name;
```

➤ Example

```
Time now;
```

- Creates a variable named **now** of type **Time**. This variable has three components (members): **hours**, **minutes** and **seconds**

```
Time dawn;
```

- It creates another variable named **dawn** of type **Time**. This variable also has three members namely **hours**, **minutes** and **seconds**

Accessing member of structure variable

6

- **Member access operator (.)** is used to access members of a struct.

- Syntax:

`struct_variable.member`

- Example:

`now.hours`

- The above syntax will access the member named **hours** of variable **now** of type **Time**.
- Similarly, all members of struct can be accessed

Initializing variable of struct

7

- We can assign value, take input and print using the same member access operator
- Example:

```
int main()
{
    Time now;
    now.hours = 5;
    now.minutes = 15;
    now.seconds = 45;
}
```

- The above code declares a variable now of type Time and initializes its data members

Structure of Library datatype

8

ISBN	Book Name	Author Name	Publisher	No. of Copies	Year of Publish
1293	Network Security	Martin	Waley	4	1998
9382	Data mining	Muhammad Zaki	Wrox	6	2003
9993	Data warehousing	Stephen Brobst	MIT	8	2003
3423	C Programming	M. Kamber	Waley	4	1996

```
struct Library
```

```
{
```

```
    int ISBN, copies, PYear;
```

```
    char bookName[30], AuthorName[30], PublisherName[30];
```

```
};
```


Example: Accessing structure variable

9

```
int main()
{
    Library libraryVariable;
    cin >> libraryVariable.ISBN;
    cin >> libraryVariable.bookName;
    cin >> libraryVariable.AuthorName;

    cout << libraryVariable.ISBN
         << libraryVariable.bookName
         << libraryVariable.AuthorName;

    int tempISBN = libraryVariable.ISBN + 1;
}
```

Accessing Structures

10

```
cout << bookName;
```

- Error! // bookName is not a variable. It is only the name of a member in a structure

```
cout << Library.bookName;
```

- Error! // Library is not the name of a variable. It is the name of a type

Common Errors in Accessing Structures (contd.)

11

```
cout << libraryVariable;
```

- `//cout` does not know how to handle the variable `libraryVariable`, as it is not one of the built-in types. You must give it individual member of `libraryVariable` that it can recognize and handle

```
cout << libraryVariable.ISBN  
      << libraryVariable.bookName;
```

- `//This is OK`

Accessing Structure Variables (Example 1)

12

```
int main(int)
{
    struct Library
    {
        int ISBN, copies, PYear;
        char bookName[30], AuthorName[30], PublisherName[30];
    };
    Library libraryVariable;

    libraryVariable.ISBN = 2321;
    strcpy(libraryVariable.bookName, "Network Security");
    strcpy(libraryVariable.AuthorName, "Martin");
    strcpy(libraryVariable.PublisherName, "Waley");
    libraryVariable.copies = 4;
    libraryVariable.PYear = 1998;

    cout << libraryVariable.ISBN << libraryVariable.bookName
         << libraryVariable.AuthorName << libraryVariable.PublisherName
         << libraryVariable.copies << libraryVariable.PYear;
}
```

Multiple struct variables

13

```
void main(void)
{
    struct Library
    {
        int ISBN, copies, PYear;
        char bookName[30], AuthorName[30], PublisherName[30];
    };
    Library libraryVariable1, libraryVariable2, libraryVariable3,
        libraryVariable4;

    Library LibraryArray[4]; // alternative and easiest way
}
```

Operations on struct variable

14

- The value of a structure variable can be assigned to another structure variable of the same type, e.g :

```
Library libraryVariable1, libraryVariable2;  
strcpy(libraryVariable1.bookName, "C Programming");  
libraryVariable1.ISBN = 2321;  
libraryVariable2 = libraryVariable1;  
cout << libraryVariable2.bookName  
      << libraryVariable2.ISBN;
```

- **Assignment** is the only operation permitted on a structure. We can not **add**, **subtract**, **multiply** or **divide** structures.

Structures variable within structures

15

```
int main()
{
    struct University
    {
        char Name[30];
        char city[30];
        Library libraryVariable;
    };
    University universityVariable;
}
```

Accessing structure variable in structure

16

```
int main()
{
    struct University
    {
        char Name[30];
        char city[30];
        Library libraryVariable;
    };
    University universityVariable;
    strcpy(universityVariable.Name, "FAST-NU");
    strcpy(universityVariable.city, "CFD");
    universityVariable.libraryVariable.ISBN = 2321;
    strcpy(universityVariable.libraryVariable.bookName, "C
programming");
    cin.get(universityVariable.libraryVariable.AuthorName,29);
    cin.get(universityVariable.libraryVariable.PublisherName,29);
}
```


Passing Structure Variables as Parameters

- An individual structure member may be passed as a parameter to a function, e.g. :

```
validLibraryData (libraryVariable.ISBN);
```

- An entire structure variable may be passed , e.g. :

```
validLibraryData (libraryVariable);
```

Example: Passing a Structure Member

18

```
void printISBN(int);  
int main()  
{  
    //assuming that Library structure has already defined  
    Library libraryVariable;  
    cin >> libraryVariable.ISBN;  
    printISBN(libraryVariable.ISBN);  
}  
  
void printISBN(int ISBN)  
{  
    cout << "ISBN = " << ISBN;  
}
```

Example: Passing an entire Structure

19

```
void printLibraryData(Library var1);
void main(void)
{
    Library libraryVariable;
    libraryVariable.ISBN = 2321;
    strcpy(libraryVariable.bookName,
            "Object Oriented Programming");
    strcpy(libraryVariable.AuthorName, "Robert");
    printLibraryData(libraryVariable);
}
void printLibraryData(Library var1)
{
    cout << "ISBN = " << var1.ISBN << "\n";
    cout << "Book name = " << var1.bookName << "\n";
}
```

Returning a Structure Variable

20

```
Library inputBookInfo();
int main()
{
    Library libraryVariable1;
    libraryVariable1 = inputBookInfo();
    cout << libraryVariable1.ISBN << libraryVariable1.bookName;
}
Library inputBookInfo()
{
    Library var1;
    var1.ISBN = 2321;
    strcpy(var1.bookName, "Object Oriented Programming");
    strcpy(var1.AuthorName, "Robert");
    return var1;
}
```

Pointers to structure variables

21

- Pointers of structure variables can be declared like pointers to any basic data type

```
Library var1, *ptrToLibrary;  
ptrToLibrary = &var1;
```

- Members of a pointer structure type variable can be accessed using (*) operator

```
(*ptrToLibrary).ISBN =2321;  
strcpy( (*ptrToLibrary).bookName,  
        "Object Oriented Programming");
```

Pointers to structure variables

22

- A more convenient way to access members of a pointer variable instead of `(*.)` operator is `(->)` operator

```
ptrToLibrary->ISBN =2321;  
strcpy( ptrToLibrary->bookName,  
        "Object Oriented Programming");
```

Pass by reference structure variables to functions

```
void Function1(Library *libPtr);
int main()
{
    Library var1;
    Function1(&var1);
    cout << var1.ISBN << var1.bookName << var1.AuthorName;
}
void Function1(Library *libraryVariable)
{
    libraryVariable->SBN = 2321;
    strcpy(libraryVariable->bookName,
           "Object Oriented Programming");
    strcpy(libraryVariable->AuthorName, "Robert");
    strcpy(libraryVariable->PublisherName, "SAMS");
    libraryVariable->copies = 4;
    libraryVariable->PYear = 1998;
}
```

Array of structure

24

```
void main(void)
{
    Library libraryArray[4];
    libraryArray[0].ISBN = 2321;
    strcpy(libraryArray[0].bookName, "Object Oriented Programming");
    libraryArray[1].ISBN = 6552;
    strcpy(libraryArray[1].bookName, "C Programming");
    libraryArray[2].ISBN = 1259;
    strcpy(libraryArray[2].bookName, "Software Engineering");

    cout << libraryArray[0].ISBN << libraryArray[1].ISBN
         << libraryArray[2].ISBN;

    cout << libraryArray[0].bookName << libraryArray[1].bookName
         << libraryArray[2].bookName;
}
```


Dynamic Memory Allocation (DMA) of structure type variables

- We can also dynamically allocate the memory of any structure type variable using `new` operator
- For example.

```
Library *ptrToLibrary;  
ptrToLibrary = new Library;
```

- Very similar to

```
float *ptrToFloat;  
ptrToFloat = new float;
```

- We can delete memory allocated at execution time using `delete`

```
delete PtrToLibrary;
```

Union

26

- As structures, unions are also used to group a number of different variables together
- The difference between union and structure is that:
 - Structure treat each of its member as a different memory location store in the main memory
 - Union treat each of its member as a single memory location store in the main memory
- i.e. All of the members of union shares a common memory of union member.

Union Example

27

```
union searchOption
```

```
{
```

```
    int SearchByRollNumber;
```

```
    char SearchByName[90];
```

```
    char SearchByAddress[90];
```

```
    char SearchByPhoneNumber[90];
```

```
};
```

```
searchOption sv;
```

```
int main()
```

```
{
```

```
    int option = 0;
```

```
    switch (option)
```

```
    {
```

```
        case 0: FunSearchRoll(sv.SearchByRollNumber); break;
```

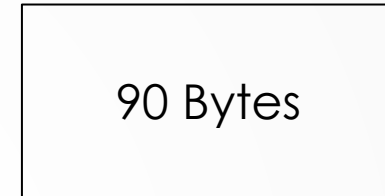
```
        case 1: FunSearchName(sv.SearchByName); break;
```

```
        case 2: FunSearchByAddress(sv.SearchByAddress); break;
```

```
        case 3: FunSearchByPhone(sv.SearchByPhoneNumber); break;
```

```
    }
```

```
}
```



Union Example

28

```
union myUnion
{
    int a; //cannot use both a and b at once
    char b;
}foo;
struct myStruct
{
    int a; //can use both a and b at once
    char b;
};
union myUnion test;
test.a = 3; //OK
test.b = 'c'; //This will change the value of a

struct myStruct s;
s.a = 3; //OK
s.b = 'c'; //OK
```

When to Use UNION?

29

- We can use the unions in the following locations
- Share a single memory location for a variable and use the same location for another variable of different data type
- Use it if you want to use, for example, a long variable as two short type variables
- We don't know what type of data is to be passed to a function, and you pass union which contains all the possible data types

Struct example

30

```
struct PersonRec
{
    string lastName;
    string firstName;
    int age;
};
typedef PersonRec PeopleArrayType[10]; //an array of 10 structs
void main(void)
{
    PeopleArrayType people; //a variable of the array type
    for (int i = 0; i < 10; i++)
    {
        cout << "Enter first name: ";
        cin >> people[i].firstName;
        cout << "Enter last name: ";
        cin >> people[i].lastName;
        cout << "Enter age: ";
        cin >> people[i].age;
    }
    for (int i = 0; i < 10; i++)
    {
        cout << people[i].firstName << ' ' << people[i].lastName
            << setw(10) << people[i].age;
    }
}
```

Questions

31

